

# MODULE 4

# Classification

Classification is the task of assigning objects to one of several predefined categories

Example:

- Determining spam email messages based upon the message header and content
- Classifying galaxies based upon their shapes
- A bank loan officer wants to analyze the data in order to know which customer are risky or safe (loan applicant)
- A marketing manager at a company needs to analyze a customer with a given profile, who will buy a computer or not

## 4.1 Classification- Definition

Classification is a task of learning a *target function*  $f$  that maps each attribute set  $x$  to one of the predefined class labels  $y$

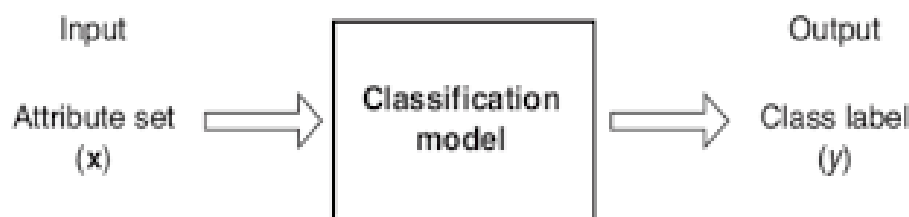


Figure 4.2. Classification as the task of mapping an input attribute set  $x$  into its class label  $y$ .

| <i>Tid</i> | Refund | Marital Status | Taxable Income | Cheat |
|------------|--------|----------------|----------------|-------|
| 1          | Yes    | Single         | 125K           | No    |
| 2          | No     | Married        | 100K           | No    |
| 3          | No     | Single         | 70K            | No    |
| 4          | Yes    | Married        | 120K           | No    |
| 5          | No     | Divorced       | 95K            | Yes   |
| 6          | No     | Married        | 60K            | No    |
| 7          | Yes    | Divorced       | 220K           | No    |
| 8          | No     | Single         | 85K            | Yes   |
| 9          | No     | Married        | 75K            | No    |
| 10         | No     | Single         | 90K            | Yes   |

One of the attributes is the **class attribute**  
In this case: Cheat

Two **class labels** (or **classes**): Yes (1), No (0)

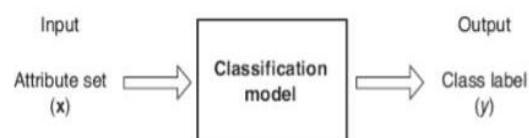


Figure 4.2. Classification as the task of mapping an input attribute set  $x$  into its class label  $y$ .

A classification model is useful for the following purposes:

- Descriptive modeling
- Predictive Modeling

### Descriptive modeling

A classification model can serve as an explanatory tool to distinguish between objects of different classes

| Name       | Body Temperature | Skin Cover | Gives Birth | Aquatic Creature | Aerial Creature | Has Legs | Hibernates | Class     |
|------------|------------------|------------|-------------|------------------|-----------------|----------|------------|-----------|
| Human      | Warm             | hair       | yes         | no               | no              | yes      | no         | Mammal    |
| Python     | Cold             | scales     | no          | no               | no              | no       | yes        | Reptile   |
| Salmon     | Cold             | scales     | no          | yes              | no              | no       | no         | Fish      |
| Whale      | Warm             | hair       | yes         | yes              | no              | no       | no         | Mammal    |
| Frog       | Cold             | none       | no          | semi             | no              | yes      | yes        | Amphibian |
| Komodo     | Cold             | scales     | no          | no               | no              | yes      | no         | Reptile   |
| Bat        | Warm             | hair       | yes         | no               | yes             | yes      | yes        | Mammal    |
| Pigeon     | Warm             | feathers   | no          | no               | yes             | yes      | no         | Bird      |
| Cat        | Warm             | fur        | yes         | no               | no              | yes      | no         | Mammal    |
| Leopard    | Cold             | scales     | yes         | yes              | no              | no       | no         | Fish      |
| Turtle     | Cold             | scales     | no          | semi             | no              | yes      | no         | Reptile   |
| Penguin    | Warm             | feathers   | no          | semi             | no              | yes      | no         | Bird      |
| Porcupine  | Warm             | quills     | yes         | no               | no              | yes      | yes        | Mammal    |
| Eel        | Cold             | scales     | no          | yes              | no              | no       | no         | Fish      |
| Salamander | Cold             | none       | no          | semi             | no              | yes      | yes        | Amphibian |

Table 4.1 Vertebrate classification

### Predictive modeling

A classification model can also be used to predict the class label of unknown records as shown below

| Name         | Body Temperature | Skin Cover | Gives Birth | Aquatic Creature | Aerial Creature | Has Legs | Hibernates | Class |
|--------------|------------------|------------|-------------|------------------|-----------------|----------|------------|-------|
| Gila Monster | cold             | scale      | no          | no               | no              | yes      | yes        | ?     |

→ A classification model can be treated as a black box that automatically assigns the class label when presented with an attribute set of an unknown record

→ Suppose we are given the characteristics of a creature known as Gila Monster

→ We can use a classification model built from the data set given in Table 4.1 to determine the class to which the creature belongs

### 4.2 General Approach for solving a classification problem

A classification technique (or classifier) is a systematic approach to building classification models from an input data set.

The model generated by a learning algorithm should understand the input data well and correctly predict the class labels of records it has never seen before.

Therefore, a key objective of the learning algorithm is to build models with good generalization capability; i.e., models that accurately predict the class labels of previously unknown records.

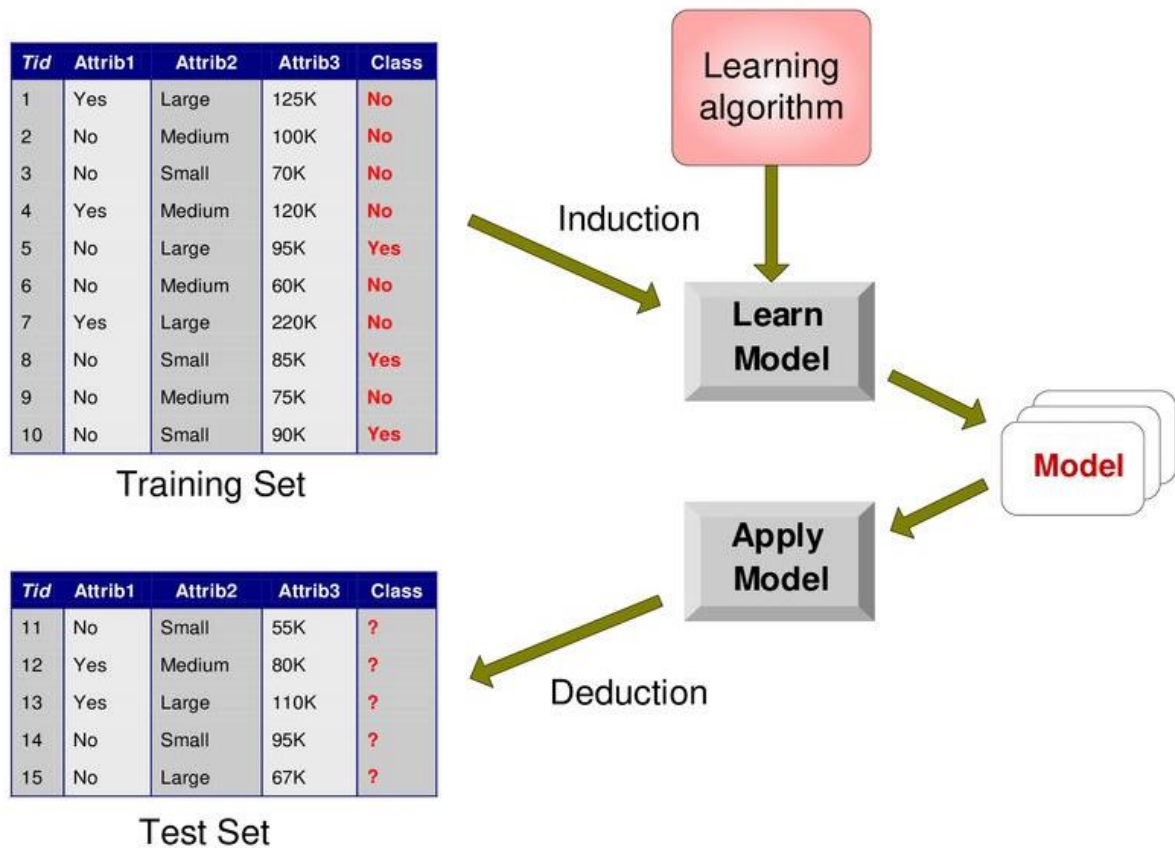


Figure 4.1 General Approach for building a classification model

Figure 4.2 shows a general approach for solving classification problems.

First, a training set consisting of records whose class labels are known must be provided.

The training set is used to build a classification model, which is subsequently applied to the test set, which consists of records with unknown class labels.

Evaluation of the performance of a classification model is based on the counts of test records correctly and incorrectly predicted by the model. These counts are tabulated in a table known as a confusion matrix.

### Confusion matrix

A table used to describe the performance of the classification model on a set of test data for which the true values are known

|              |                  | Predicted Class  |                  |
|--------------|------------------|------------------|------------------|
|              |                  | <i>Class = 1</i> | <i>Class = 0</i> |
| Actual Class | <i>Class = 1</i> | $f_{11}$         | $f_{10}$         |
|              | <i>Class = 0</i> | $f_{01}$         | $f_{00}$         |

Table 4.2 Confusion matrix for a 2-class problem

Table 4.2 depicts the confusion matrix for a binary classification problem. Each entry  $f_{ij}$  in this table denotes the number of records from class  $i$  predicted to be of class  $j$ . For instance,  $f_{01}$  is the number of records from class 0 incorrectly predicted as class 1.

Based on the entries in the confusion matrix, the total number of correct predictions made by the model is  $(f_{11} + f_{00})$  and the total number of incorrect predictions is  $(f_{10} + f_{01})$ .

$$\text{Accuracy} = \frac{\# \text{ correct predictions}}{\text{total \# of predictions}} = \frac{f_{11} + f_{00}}{f_{11} + f_{10} + f_{01} + f_{00}}$$

$$\text{Error rate} = \frac{\# \text{ wrong predictions}}{\text{total \# of predictions}} = \frac{f_{10} + f_{01}}{f_{11} + f_{10} + f_{01} + f_{00}}$$

### 4.3 Types of Classifiers

Decision tree classifier

Rule based classifier

Nearest Neighbor classifier

Bayesian Classifier

#### 4.3.1 Decision Tree Induction

Decision tree builds classification or regression models in the form of a tree structure. It breaks down a dataset into smaller and smaller subsets while at the same time an associated decision tree is incrementally developed. The final result is a tree with decision nodes and leaf nodes

How a Decision Tree works?

Figure 4.2 shows the decision tree for the mammal classification problem.

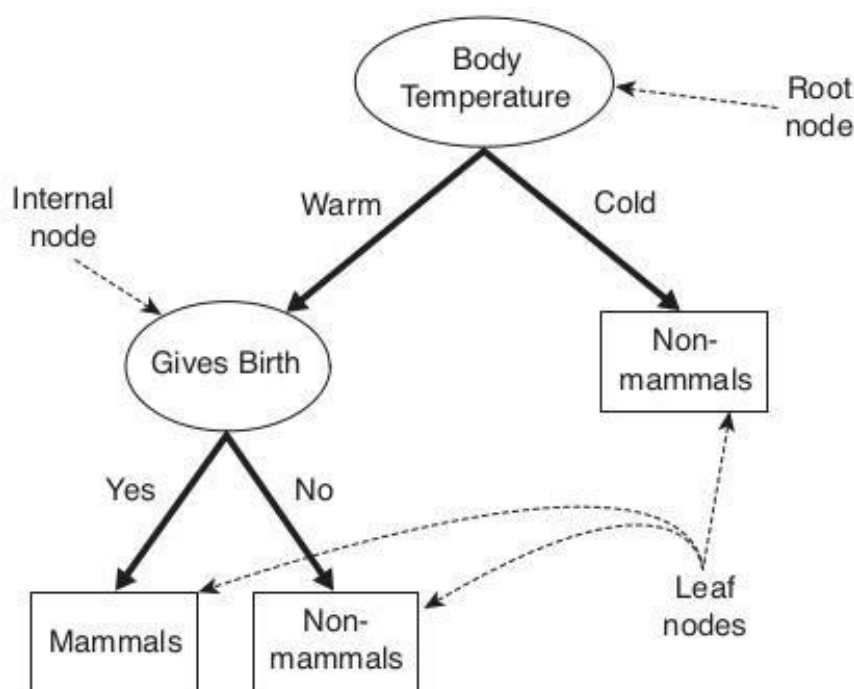


Figure 4.2: A Decision tree for the mammal classification problem

The tree has three types of nodes:

Root node

Internal nodes

Leaf or terminal nodes

Root node: Has no incoming edges and zero or more outgoing edges

Internal node: each of which has exactly one incoming edge and two or more outgoing edges

Leaf or terminal nodes: each of which has exactly one incoming edge and no outgoing edges

→ Each leaf node is assigned a class label

→ Non terminal nodes (root or internal nodes) contains attribute test condition

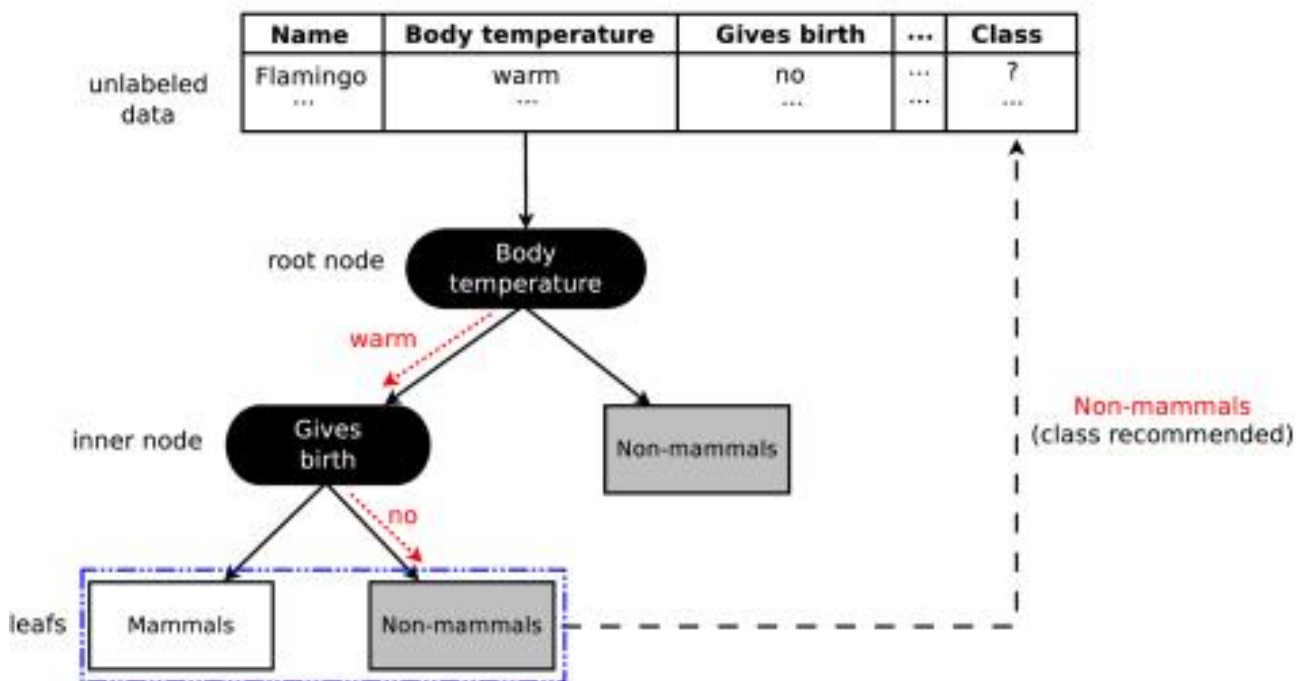


Figure 4.3: Classifying an unlabelled vertebrate

### How to build a decision tree

- Many decision trees can be constructed from a given set of attributes
- Some of the trees are more accurate than others
- Hunts algorithm can be used

### Hunts Algorithm

Hunts algorithm grows a decision tree in a recursive fashion by partitioning the training records into successively purer subsets

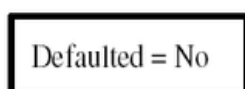
Let  $D_t$  be the set of training records that reach a node  $t$

-If  $D_t$  contains records that belong the same class  $y_t$ , then  $t$  is a leaf node labeled as  $y_t$

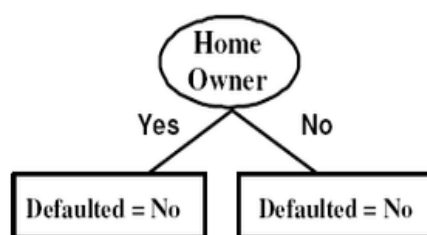
-If Dt contains records that belong to more than one class, use an attribute test to split the data into smaller subsets. Recursively apply the procedure to each subset

Consider the following example of predicting whether loan applicant will repay the loan

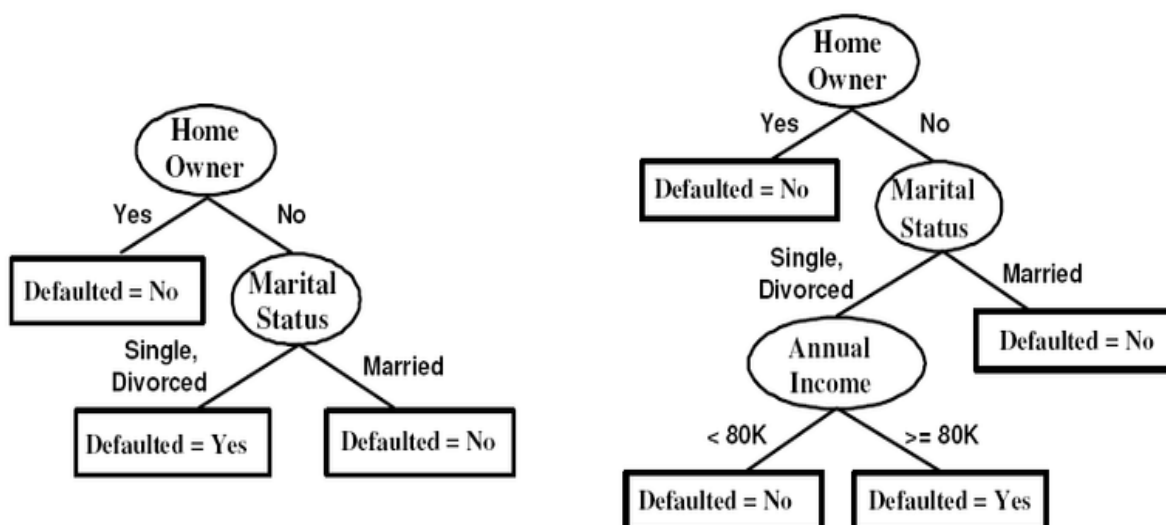
| Tid | Home Owner | Marital Status | Annual Income | Defaulted Borrower |
|-----|------------|----------------|---------------|--------------------|
| 1   | Yes        | Single         | 125K          | No                 |
| 2   | No         | Married        | 100K          | No                 |
| 3   | No         | Single         | 70K           | No                 |
| 4   | Yes        | Married        | 120K          | No                 |
| 5   | No         | Divorced       | 95K           | Yes                |
| 6   | No         | Married        | 60K           | No                 |
| 7   | Yes        | Divorced       | 220K          | No                 |
| 8   | No         | Single         | 85K           | Yes                |
| 9   | No         | Married        | 75K           | No                 |
| 10  | No         | Single         | 90K           | Yes                |



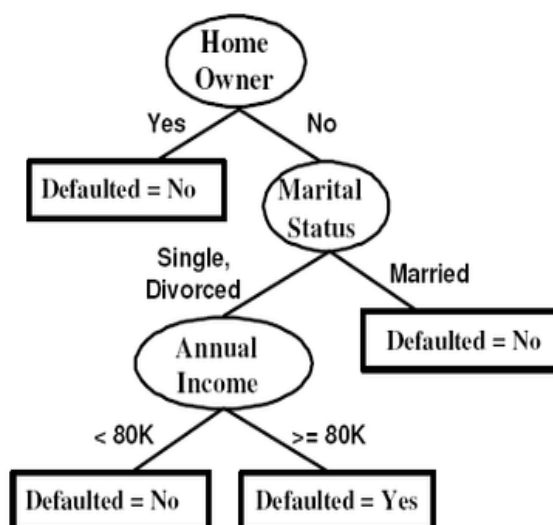
(a) Step 1



(b) Step 2



(c) Step 3

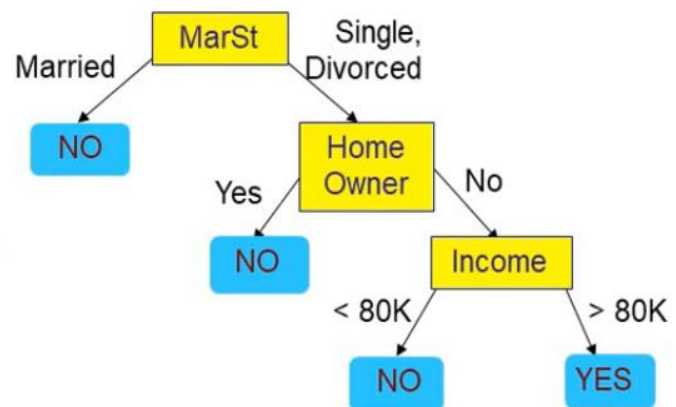


(d) Step 4

There could be more than one tree that fits the same data as shown in the Figure below

categorical      categorical      continuous      class

| ID | Home Owner | Marital Status | Annual Income | Defaulted Borrower |
|----|------------|----------------|---------------|--------------------|
| 1  | Yes        | Single         | 125K          | No                 |
| 2  | No         | Married        | 100K          | No                 |
| 3  | No         | Single         | 70K           | No                 |
| 4  | Yes        | Married        | 120K          | No                 |
| 5  | No         | Divorced       | 95K           | Yes                |
| 6  | No         | Married        | 60K           | No                 |
| 7  | Yes        | Divorced       | 220K          | No                 |
| 8  | No         | Single         | 85K           | Yes                |
| 9  | No         | Married        | 75K           | No                 |
| 10 | No         | Single         | 90K           | Yes                |

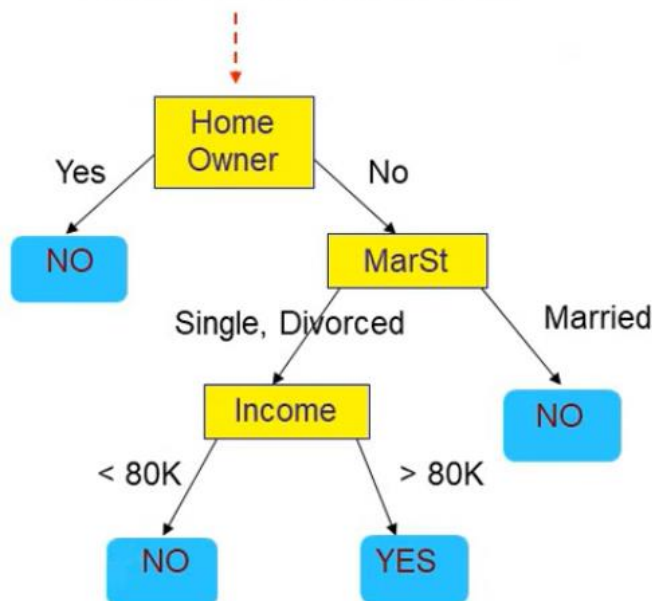


There could be more than one tree that fits the same data!

## Apply Model to Test Data

Test Data

Start from the root of tree.



| Home Owner | Marital Status | Annual Income | Defaulted Borrower |
|------------|----------------|---------------|--------------------|
| No         | Married        | 80K           | ?                  |

Above test data is classified as Defaulted borrower is NO.

Methods for Expressing attribute test conditions:

Decision tree induction algorithm must provide a method for expressing an attribute test condition and its corresponding outcomes for different attributes types.

1. Binary Attributes: The test condition for a binary attribute generates two potential outcomes, as shown in Figure 4.4

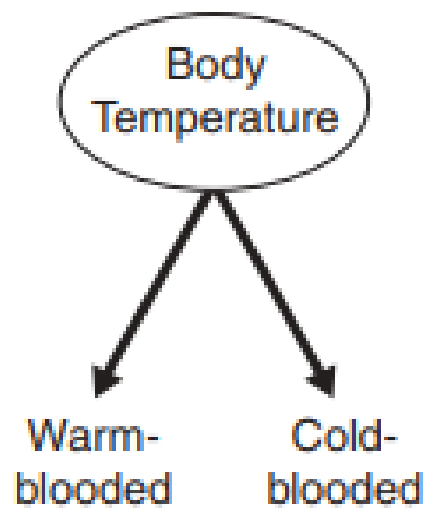


Figure 4.4 Test condition for binary attribute

2. Nominal Attributes: Since a nominal attribute can have many values, its test condition can be expressed in two ways, as shown in Figure 4.5.

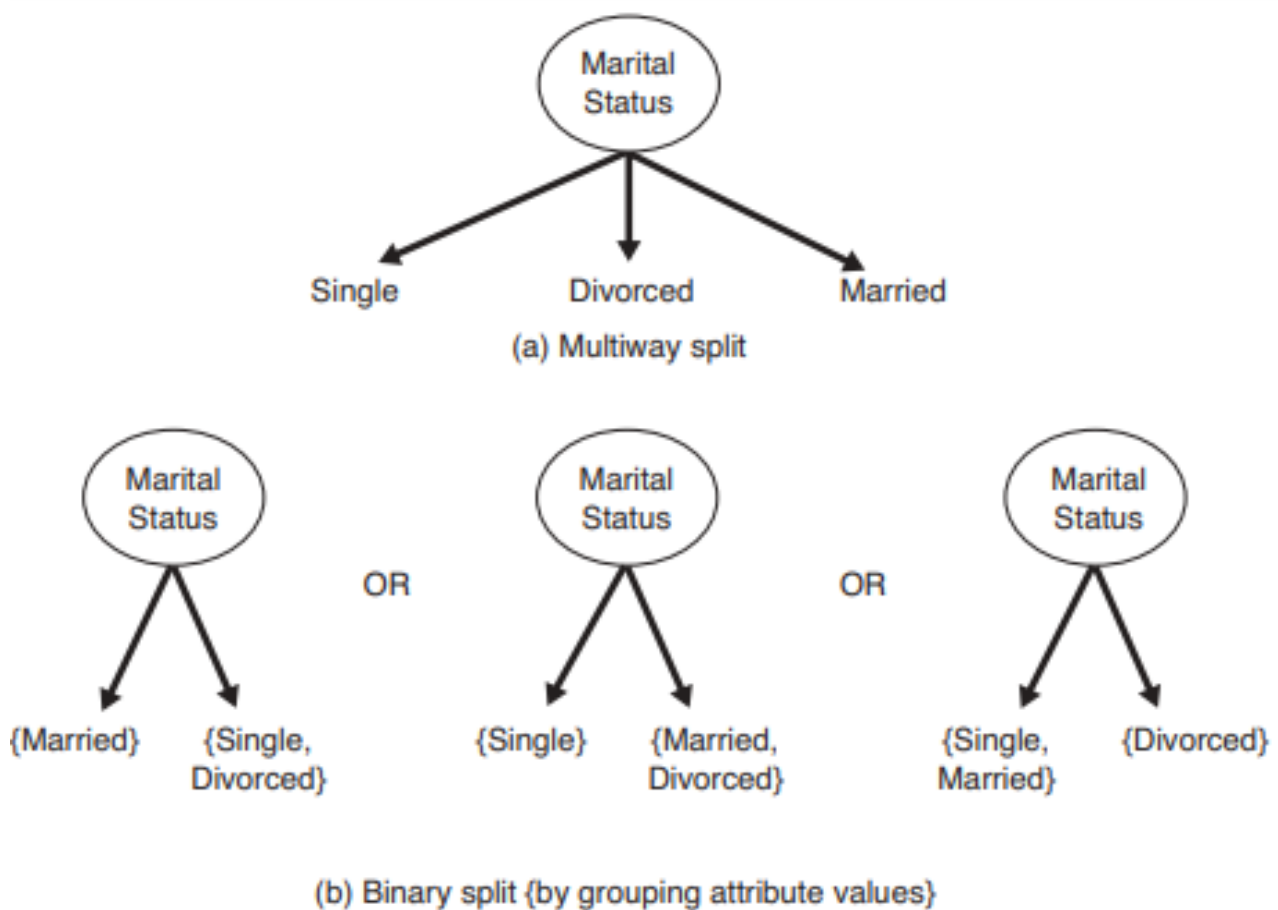


Figure 4.5: Test condition for nominal attribute

3. Ordinal Attribute

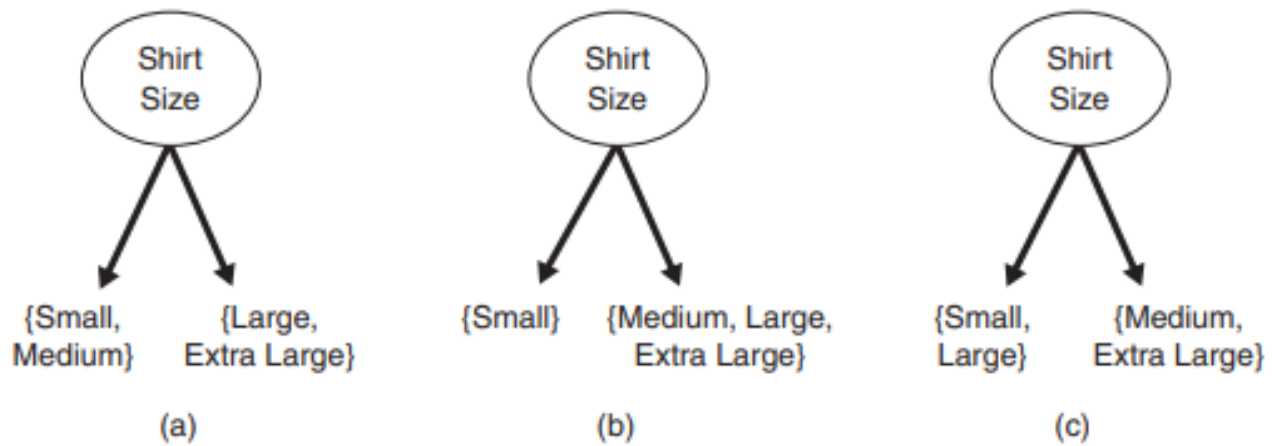


Figure 4.6: Different ways of grouping ordinal attribute values

Ordinal attributes can also produce binary or multiway splits

Ordinal attribute values can be grouped as long as the grouping does not violate the order property of the attribute values.

Figure 4.6 illustrates various ways of splitting training records based on the Shirt Size attribute. The groupings shown in Figures 4.6(a) and (b) preserve the order among the attribute values, whereas the grouping shown in Figure 4.6(c) violates this property because it combines the attribute values Small and Large into the same partition while Medium and Extra Large are combined into another partition.

4. Continuous Attributes: For continuous attributes, the test condition can be expressed as a comparison test or with binary outcomes

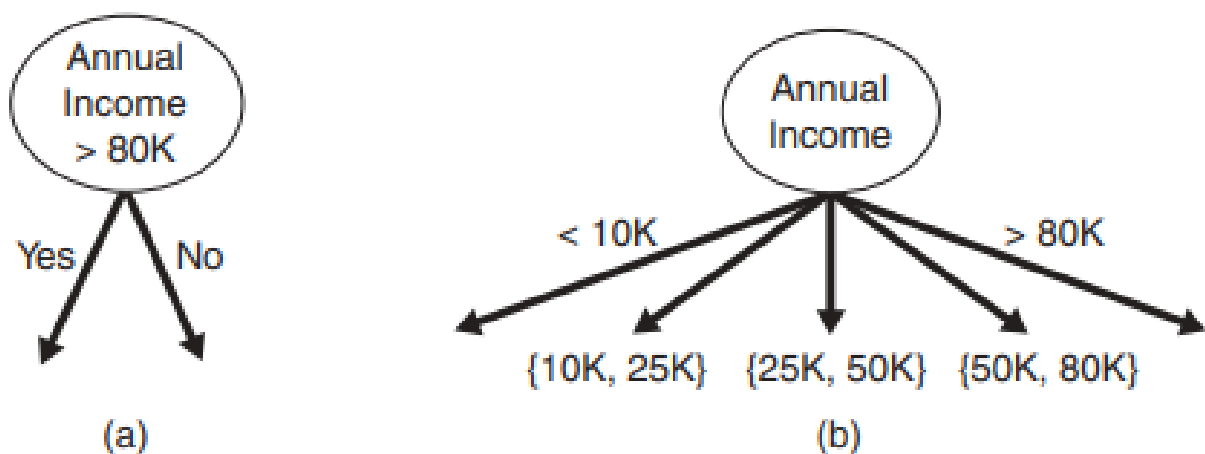


Figure 4.7 Test condition for continuous attribute

### Measures for selecting the best split

- There are many measures that can be used to determine the best way to split the records
- The measures developed for selecting the best split are often based on the degree of impurity of the child node.
- Smaller the degree of impurity the more biased the class distribution is.
- A node with class distribution(0,1) has zero impurity, whereas a node with uniform distribution (0.5, 0.5) has the highest impurity

→  $P(i|t)$ - denote the fraction of records belonging to class  $i$  at a given node  $t$ .

### Example of impurity measures

$$\text{Entropy}(t) = -\sum_{i=1}^c p(i|t) \log p(i|t)$$

$$\text{Gini}(t) = 1 - \sum_{i=1}^c [p(i|t)]^2$$

$$\text{Classification error}(t) = 1 - \max_i [p(i|t)]$$

Where  $c$  is the number of classes

$0 \log 0 = 0$  in entropy calculations

| Node $N_1$ | Count |
|------------|-------|
| Class=0    | 0     |
| Class=1    | 6     |

$$\text{Gini} = 1 - (0/6)^2 - (6/6)^2 = 0$$

$$\text{Entropy} = -(0/6) \log_2(0/6) - (6/6) \log_2(6/6) = 0$$

$$\text{Error} = 1 - \max[0/6, 6/6] = 0$$

| Node $N_2$ | Count |
|------------|-------|
| Class=0    | 1     |
| Class=1    | 5     |

$$\text{Gini} = 1 - (1/6)^2 - (5/6)^2 = 0.278$$

$$\text{Entropy} = -(1/6) \log_2(1/6) - (5/6) \log_2(5/6) = 0.650$$

$$\text{Error} = 1 - \max[1/6, 5/6] = 0.167$$

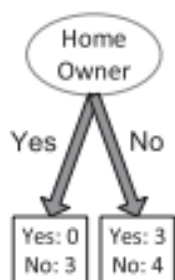
| Node $N_3$ | Count |
|------------|-------|
| Class=0    | 3     |
| Class=1    | 3     |

$$\text{Gini} = 1 - (3/6)^2 - (3/6)^2 = 0.5$$

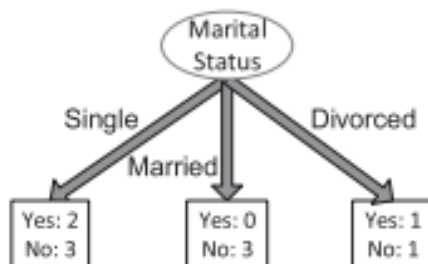
$$\text{Entropy} = -(3/6) \log_2(3/6) - (3/6) \log_2(3/6) = 1$$

$$\text{Error} = 1 - \max[3/6, 3/6] = 0.5$$

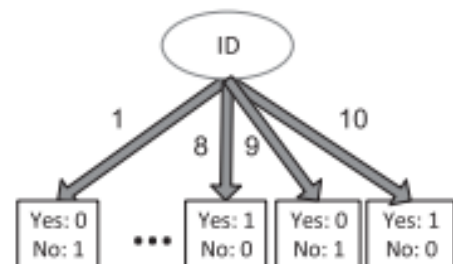
Consider the candidate attribute test condition shown in Figures 4.8(a) and (b) for the loan borrower classification problem (Table 4.2: Data sample for loan borrower classification problem)



(a)



(b)



(c)

Figure 4.8: Examples of candidate attribute test conditions

| ID | Home Owner | Marital Status | Annual Income | Defaulted? |
|----|------------|----------------|---------------|------------|
| 1  | Yes        | Single         | 125000        | No         |
| 2  | No         | Married        | 100000        | No         |
| 3  | No         | Single         | 70000         | No         |
| 4  | Yes        | Married        | 120000        | No         |
| 5  | No         | Divorced       | 95000         | Yes        |
| 6  | No         | Single         | 60000         | No         |
| 7  | Yes        | Divorced       | 220000        | No         |
| 8  | No         | Single         | 85000         | Yes        |
| 9  | No         | Married        | 75000         | No         |
| 10 | No         | Single         | 90000         | Yes        |

Table 4.2: Data sample for loan borrower classification problem

Entropy can be calculated as follows:

$$I(\text{Home Owner} = \text{yes}) = -\frac{0}{3} \log_2 \frac{0}{3} - \frac{3}{3} \log_2 \frac{3}{3} = 0$$

$$I(\text{Home Owner} = \text{no}) = -\frac{3}{7} \log_2 \frac{3}{7} - \frac{4}{7} \log_2 \frac{4}{7} = 0.985$$

$$I(\text{Home Owner}) = \frac{3}{10} \times 0 + \frac{7}{10} \times 0.985 = 0.690$$

$$I(\text{Marital Status} = \text{Single}) = -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} = 0.971$$

$$I(\text{Marital Status} = \text{Married}) = -\frac{0}{3} \log_2 \frac{0}{3} - \frac{3}{3} \log_2 \frac{3}{3} = 0$$

$$I(\text{Marital Status} = \text{Divorced}) = -\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2} = 1.000$$

$$I(\text{Marital Status}) = \frac{5}{10} \times 0.971 + \frac{3}{10} \times 0 + \frac{2}{10} \times 1 = 0.686$$

Thus, Marital Status has a lower weighted entropy than Home Owner.

To determine how well the test condition performs, we need to compare the degree of impurity of parent node (before splitting) with the degree of impurity of the child node (after splitting)

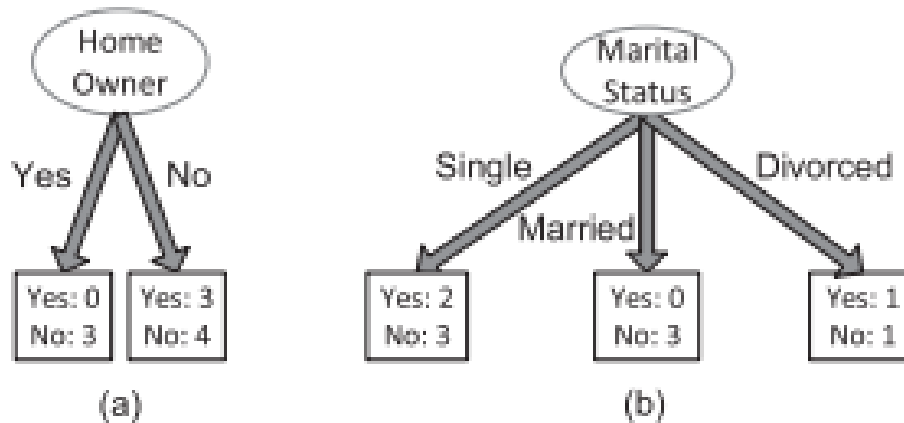
Larger the difference, better the test condition

Gain is the criteria used to determine the goodness of the split

$$\Delta = I(\text{parent}) - I(\text{children}),$$

When entropy is used as the impurity measure, the difference in entropy is commonly known as information gain,  $\Delta_{\text{info}}$ .

### Example



$$I(\text{parent}) = -\frac{3}{10} \log_2 \frac{3}{10} - \frac{7}{10} \log_2 \frac{7}{10} = 0.881$$

The information gains for Home Owner and Marital Status are each given by

$$\begin{aligned} \Delta_{\text{info}}(\text{Home Owner}) &= 0.881 - 0.690 = 0.191 \\ \Delta_{\text{info}}(\text{Marital Status}) &= 0.881 - 0.686 = 0.195 \end{aligned}$$

The information gain for Marital Status is thus higher due to its lower weighted entropy, which will thus be considered for splitting.

### Gain ratio

Used to compensate for attributes that produce a large number of child nodes.

$$\text{Gain ratio} = \frac{\Delta_{\text{info}}}{\text{Split Info}}$$

$$\text{Split info} = - \sum_{i=1}^k \frac{N(v_i)}{N} \log_2 \frac{N(v_i)}{N}$$

Where  $N(v_i)$  is the number of instances assigned to node  $v_i$  and  $k$  is the total number of splits.

**Example:**

Consider the data set given below

We want to select the best attribute test condition among the following three attributes: Gender, Car Type, and Customer ID.

| Customer ID | Gender | Car Type | Shirt Size  | Class |
|-------------|--------|----------|-------------|-------|
| 1           | M      | Family   | Small       | C0    |
| 2           | M      | Sports   | Medium      | C0    |
| 3           | M      | Sports   | Medium      | C0    |
| 4           | M      | Sports   | Large       | C0    |
| 5           | M      | Sports   | Extra Large | C0    |
| 6           | M      | Sports   | Extra Large | C0    |
| 7           | F      | Sports   | Small       | C0    |
| 8           | F      | Sports   | Small       | C0    |
| 9           | F      | Sports   | Medium      | C0    |
| 10          | F      | Luxury   | Large       | C0    |
| 11          | M      | Family   | Large       | C1    |
| 12          | M      | Family   | Extra Large | C1    |
| 13          | M      | Family   | Medium      | C1    |
| 14          | M      | Luxury   | Extra Large | C1    |
| 15          | F      | Luxury   | Small       | C1    |
| 16          | F      | Luxury   | Small       | C1    |
| 17          | F      | Luxury   | Medium      | C1    |
| 18          | F      | Luxury   | Medium      | C1    |
| 19          | F      | Luxury   | Medium      | C1    |
| 20          | F      | Luxury   | Large       | C1    |

Table 4.3:Data set

$$\text{Entropy}(\text{parent}) = -\frac{10}{20} \log_2 \frac{10}{20} - \frac{10}{20} \log_2 \frac{10}{20} = 1.$$

If Gender is used as attribute test condition:

$$\begin{aligned} \text{Entropy}(\text{children}) &= \frac{10}{20} \left[ -\frac{6}{10} \log_2 \frac{6}{10} - \frac{4}{10} \log_2 \frac{4}{10} \right] \times 2 = 0.971 \\ \text{Gain Ratio} &= \frac{1 - 0.971}{-\frac{10}{20} \log_2 \frac{10}{20} - \frac{10}{20} \log_2 \frac{10}{20}} = \frac{0.029}{1} = 0.029 \end{aligned}$$

If Car Type is used as attribute test condition:

$$\begin{aligned} \text{Entropy(children)} &= \frac{4}{20} \left[ -\frac{1}{4} \log_2 \frac{1}{4} - \frac{3}{4} \log_2 \frac{3}{4} \right] + \frac{8}{20} \times 0 \\ &\quad + \frac{8}{20} \left[ -\frac{1}{8} \log_2 \frac{1}{8} - \frac{7}{8} \log_2 \frac{7}{8} \right] = 0.380 \end{aligned}$$

$$\text{Gain Ratio} = \frac{1 - 0.380}{-\frac{4}{20} \log_2 \frac{4}{20} - \frac{8}{20} \log_2 \frac{8}{20} - \frac{8}{20} \log_2 \frac{8}{20}} = \frac{0.620}{1.52} = 0.41$$

Finally, if Customer ID is used as attribute test condition:

$$\text{Entropy(children)} = \frac{1}{20} \left[ -\frac{1}{1} \log_2 \frac{1}{1} - \frac{0}{1} \log_2 \frac{0}{1} \right] \times 20 = 0$$

$$\text{Gain Ratio} = \frac{1 - 0}{-\frac{1}{20} \log_2 \frac{1}{20} \times 20} = \frac{1}{4.32} = 0.23$$

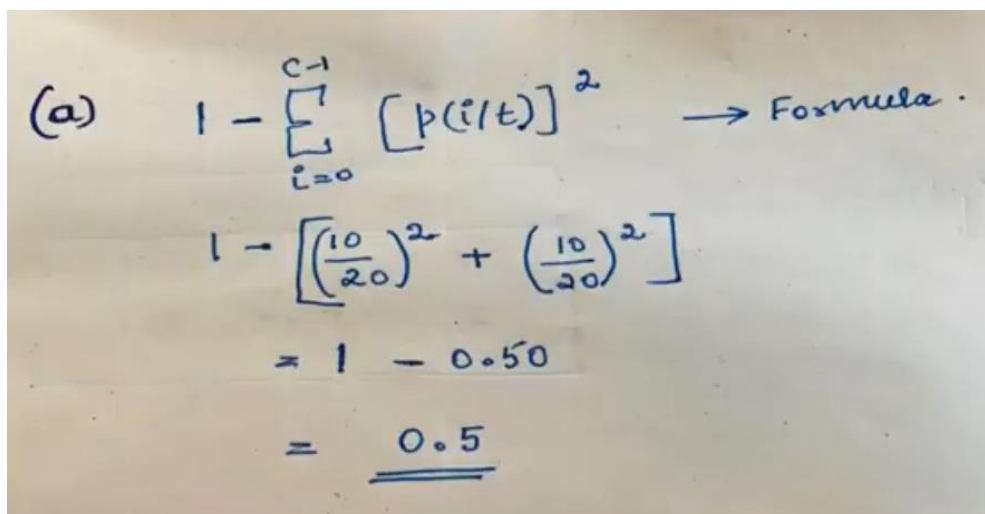
Thus, even though Customer ID has the highest information gain, its gain ratio is lower than Car Type since it produces a larger number of splits.

### Exercises

Consider the training examples shown in Table 4.3 for a binary classification problem.

- Compute the Gini index for the overall collection of training examples.
- Compute the Gini index for the **Customer ID** attribute.
- Compute the Gini index for the **Gender** attribute.
- Compute the Gini index for the **Car Type** attribute using multiway split.
- Compute the Gini index for the **Shirt Size** attribute using multiway split.
- Which attribute is better, **Gender**, **Car Type**, or **Shirt Size**?
- Explain why Customer ID should not be used as the attribute test condition even though it has the lowest Gini.

→ Compute the Gini index for the overall collection of training examples



(a)  $1 - \sum_{i=0}^{c-1} [p(i/t)]^2 \rightarrow \text{Formula}$

$$1 - \left[ \left( \frac{10}{20} \right)^2 + \left( \frac{10}{20} \right)^2 \right]$$

$$= 1 - 0.50$$

$$= \underline{\underline{0.5}}$$

→ Compute the Gini index for the **Customer ID** attribute.

The Gini for each customer ID is zero

→ Compute the Gini index for the **Gender** attribute

Gini (Male)

| Male      | Count |
|-----------|-------|
| class = 0 | 6     |
| class = 1 | 4     |

$$1 - \left[ \left( \frac{6}{10} \right)^2 + \left( \frac{4}{10} \right)^2 \right]$$

$$= 1 - [0.36 + 0.16]$$

$$= \underline{\underline{0.48}}$$

Gini (Female)

| Female    | Count |
|-----------|-------|
| class = 0 | 4     |
| class = 1 | 6     |

$$1 - \left[ \left( \frac{4}{10} \right)^2 + \left( \frac{6}{10} \right)^2 \right]$$

$$= 1 - [0.16 + 0.36]$$

$$= 0.48$$

$$\text{Overall Gini} = \frac{10}{20} \times 0.48 + \frac{10}{20} \times 0.48$$

$$= \underline{\underline{0.48}}$$

→ Compute the Gini index for the **Car Type** attribute using multiway split.

Gini (Family)

| Family    | Count |
|-----------|-------|
| class = 0 | 1     |
| class = 1 | 3     |

$$1 - \left[ \left( \frac{1}{4} \right)^2 + \left( \frac{3}{4} \right)^2 \right]$$

$$= 1 - [0.0625 + 0.5625]$$

$$= \underline{\underline{0.375}}$$

Gini (Sports)

| Sports    | Count |
|-----------|-------|
| class = 0 | 8     |
| class = 1 | 0     |

$$1 - \left[ \left( \frac{8}{8} \right)^2 + \left( \frac{0}{8} \right)^2 \right]$$

$$= 1 - 1$$

$$= \underline{\underline{0}}$$

Gini (Luxury)

| Luxury    | Count |
|-----------|-------|
| class = 0 | 1     |
| class = 1 | 7     |

$$1 - \left[ \left( \frac{1}{8} \right)^2 + \left( \frac{7}{8} \right)^2 \right]$$

$$= \underline{\underline{0.21375}}$$

$$\text{Overall Gini} = \frac{4}{20} \times 0.375 + \frac{8}{20} \times 0 + \frac{8}{20} \times 0.21375$$

$$= \underline{\underline{0.1625}}$$

→ Compute the Gini index for the **Shirt Size** attribute using multiway split.

$$Gini(\text{Small})$$

| Small     | count |
|-----------|-------|
| class = 0 | 3     |
| class = 1 | 2     |

$$1 - \left[ \left( \frac{3}{5} \right)^2 + \left( \frac{2}{5} \right)^2 \right] = \underline{0.48}$$

$$Gini(\text{Medium})$$

| Medium    | count |
|-----------|-------|
| class = 0 | 3     |
| class = 1 | 4     |

$$1 - \left[ \left( \frac{3}{7} \right)^2 + \left( \frac{4}{7} \right)^2 \right] = \underline{0.49}$$

$$Gini(\text{Large})$$

| Large     | count |
|-----------|-------|
| class = 0 | 2     |
| class = 1 | 2     |

$$1 - \left[ \left( \frac{2}{4} \right)^2 + \left( \frac{2}{4} \right)^2 \right] = 0.5$$

$$Gini(\text{Extra Large})$$

| E-Large   | count |
|-----------|-------|
| class = 0 | 2     |
| class = 1 | 2     |

$$1 - \left[ \left( \frac{2}{4} \right)^2 + \left( \frac{2}{4} \right)^2 \right] = \underline{0.5}$$

Overall Gini Index =

$$\frac{5}{20} \times 0.48 + \frac{7}{20} \times 0.49 + \frac{4}{20} \times 0.5 + \frac{4}{20} \times 0.5 = \underline{0.4915}$$

→ Which attribute is better, **Gender**, **Car Type**, or **Shirt Size**?

Car type, because it has the lowest gini compared to the three attributes

→ Explain why Customer ID should not be used as the attribute test condition even though it has the lowest Gini.

Attribute has no predictive power since the new customers are assigned to the new customer ID's

Algorithm for decision tree induction

A skeleton decision tree induction algorithm called TreeGrowth is shown in Algorithm below

The input to this algorithm consists of the training records E and the attribute set F.

The algorithm works by recursively selecting the best attribute to split the data (Step 7) and expanding the leaf nodes of the tree (step 11 and 12) until the stopping criterion is met (step 1).

**Algorithm 4.1:** A skeleton decision tree induction algorithm

TreeGrowth (E, F)

1. **if** stopping\_cond(E,F) = true **then**
2.   leaf = createNode()
3.   leaf.label = Classify(E).
4.   return leaf.
5. **else**
6.   root = createNode().
7.   root.test-cond = find\_best\_split(E, F).
8.   let  $V = \{v \mid v \text{ is a possible outcome of root.test-cond} \}$ .
9.   **for each**  $v \in V$  **do**
10.      $E_v = \{e \mid \text{root.test-cond}(e) = v \text{ and } e \in E\}$ .
11.     child = TreeGrowth( $E_v$ , F).
12.     add child as descendent of root and label the edge (root  $\rightarrow$  child) as v.
13.   **end for**
14. **end if**
15. return root.

The Details of the Algorithm are explained below:

- The createNode() function extends the decision tree by creating a new node
- The find\_best\_split() function determines which attribute should be selected as the test condition for splitting the training records. The choice of test condition depends on which impurity measure is used to determine the goodness of split. Some of them are Entropy, Gini Index etc..
- The Classify() function determines the class label to be assigned to a leaf node.
- The stopping\_cond() function is used to terminate the tree-growing process by testing whether all the records have either the same class label or the same attribute value s.

**Characteristics of decision tree induction**

- Decision tree induction is a non-parametric approach for building classification models
- Finding an optimal decision tree is an NP-complete problem. Many decision tree algorithms employ a heuristic-based approach to guide their search in the vast hypothesis space.
- Techniques developed for constructing decision trees are computationally inexpensive, making it possible to quickly construct models even when the training set size is very large.
- Decision trees, especially smaller-sized trees, are relatively easy to interpret.
- Decision trees provide an expressive representation for learning discrete-valued functions.
- Decision tree algorithms are quite robust to the presence of noise, especially when methods for avoiding over-fitting.
- The presence of redundant attributes does not adversely affect the accuracy of decision trees

→ Since most decision tree algorithms employ a top-down, recursive partitioning approach, the number of records becomes smaller as we traverse down the tree.

→ These algorithms usually employ a greedy strategy that grows a decision tree by making a series of locally optimum decisions about which attribute to use for partitioning the data

→ Many Algorithms:

-Hunts Algorithm

-CART

-ID3, C4.5

-SLIQ, SPRINT

#### 4.3.2 Rule based Classifiers

A rule based classifier is a technique for classifying records using a collection of “if ... then ...” rules.

Following table shows an example of a model generated by a rule based classifier for the vertebrate classification problem.

|                                                              |
|--------------------------------------------------------------|
| 1) R1: (Give Birth = no) ^ (Aerial Creature = yes) → Birds   |
| 2) R2: (Give Birth = no) ^ (Aquatic Creature = yes) → Fishes |
| 3) R3: (Give Birth = yes) ^ (Blood Type = warm) → Mammals    |
| 4) R4: (Give Birth = no) ^ (Aerial Creature = no) → Reptiles |
| 5) R5: (Live in Water = sometimes) → Amphibians              |

Each classification rule can be expressed in the following ways:

$$r_i: (\text{Condition}_i) \rightarrow y_i$$

$$\text{Condition}_i = (A_1 \text{ op } v_1) \wedge (A_2 \text{ op } v_2) \wedge \dots (A_k \text{ op } v_k)$$

Where  $(A_j, v_j)$  is an attribute-value pair and op is a logical operator chosen from the set  $\{=, !=, <, <=, >, >=\}$

Left hand side of the rule is called the rule **antecedent** or **precondition**

Right hand side of the rule is called the rule **consequent**, which contains the predicted class  $y_i$ .

| Name          | Blood Type | Give Birth | Can Fly | Live in Water | Class      |
|---------------|------------|------------|---------|---------------|------------|
| human         | warm       | yes        | no      | no            | mammals    |
| python        | cold       | no         | no      | no            | reptiles   |
| salmon        | cold       | no         | no      | yes           | fishes     |
| whale         | warm       | yes        | no      | yes           | mammals    |
| frog          | cold       | no         | no      | sometimes     | amphibians |
| komodo        | cold       | no         | no      | no            | reptiles   |
| bat           | warm       | yes        | yes     | no            | mammals    |
| pigeon        | warm       | no         | yes     | no            | birds      |
| cat           | warm       | yes        | no      | no            | mammals    |
| leopard shark | cold       | yes        | no      | yes           | fishes     |
| turtle        | cold       | no         | no      | sometimes     | reptiles   |
| penguin       | warm       | no         | no      | sometimes     | birds      |
| porcupine     | warm       | yes        | no      | no            | mammals    |
| eel           | cold       | no         | no      | yes           | fishes     |
| salamander    | cold       | no         | no      | sometimes     | amphibians |
| gila monster  | cold       | no         | no      | no            | reptiles   |
| platypus      | warm       | no         | no      | no            | mammals    |
| owl           | warm       | no         | yes     | no            | birds      |
| dolphin       | warm       | yes        | no      | yes           | mammals    |
| eagle         | warm       | no         | yes     | no            | birds      |

### Application of rule based classifier

R1: (Give Birth = no)  $\wedge$  (Can Fly = yes)  $\rightarrow$  Birds

R2: (Give Birth = no)  $\wedge$  (Live in Water = yes)  $\rightarrow$  Fishes

R3: (Give Birth = yes)  $\wedge$  (Blood Type = warm)  $\rightarrow$  Mammals

R4: (Give Birth = no)  $\wedge$  (Can Fly = no)  $\rightarrow$  Reptiles

R5: (Live in Water = sometimes)  $\rightarrow$  Amphibians

| Name         | Blood Type | Give Birth | Can Fly | Live in Water | Class |
|--------------|------------|------------|---------|---------------|-------|
| hawk         | warm       | no         | yes     | no            | ?     |
| grizzly bear | warm       | yes        | no      | no            | ?     |

First vertebrate triggers rule R1, hence classified as Bird

Second vertebrate triggers rule R3, hence classified as Mammals

### Rule coverage and accuracy

Coverage of a rule:

Fraction of records satisfy the antecedent of a rule

Accuracy of a rule:

Fraction of records that satisfy the antecedent that also satisfy the consequent of a rule

$$\text{Coverage}(r) = \frac{|A|}{|D|}$$

$$\text{Accuracy}(r) = \frac{|A \cap y|}{|A|}, \quad (5.3)$$

where  $|A|$  is the number of records that satisfy the rule antecedent,  $|A \cap y|$  is the number of records that satisfy both the antecedent and consequent, and  $|D|$  is the total number of records.

### Example

**(Status= Single) → No**

**Coverage=4/10= 40%**

**Accuracy=2/4=50%**

| Tid | Home Owner | Marital Status | Annual Income | Defaulted Borrower |
|-----|------------|----------------|---------------|--------------------|
| 1   | Yes        | Single         | 125K          | No                 |
| 2   | No         | Married        | 100K          | No                 |
| 3   | No         | Single         | 70K           | No                 |
| 4   | Yes        | Married        | 120K          | No                 |
| 5   | No         | Divorced       | 95K           | Yes                |
| 6   | No         | Married        | 60K           | No                 |
| 7   | Yes        | Divorced       | 220K          | No                 |
| 8   | No         | Single         | 85K           | Yes                |
| 9   | No         | Married        | 75K           | No                 |
| 10  | No         | Single         | 90K           | Yes                |

### How does a rule based classifier works

A rule-based classifier classifies a test record based on the rule triggered by the record. To illustrate how a rule-based classifier works, consider the rule set given below Table and the following vertebrates

| Name          | Blood type | Skin cover | Gives birth | Lives in water | can fly | Has legs | Hibernate |
|---------------|------------|------------|-------------|----------------|---------|----------|-----------|
| Lemur         | Warm       | Fur        | Yes         | No             | No      | Yes      | Yes       |
| Turtle        | Cold       | Scales     | No          | Semi           | No      | Yes      | No        |
| Dogfish shark | cold       | scales     | Yes         | Yes            | No      | No       | no        |

Lemur triggers rule r3, hence it is classified as a mammal

Turtle triggers both r4 and r5

Dogfish shark triggers none of the rules

### Characteristics of a rule set

Mutually Exclusive rules:

- classifier contains mutually exclusive rules if the rules are independent of each other
- every record is covered by at most one rule

### Exhaustive rules

-classifier has exhaustive coverage if there is a rule for every possible combination of attribute values

- each record is covered by at least one rules

If the rule set is not exhaustive, then a default rule

Rd: ()  $\rightarrow$  yd

Must be added to cover the remaining cases.

If the rule set is not mutually exclusive, there are two ways to overcome the problem

Ordered rules

Unordered rules

### Ordered rules

In this approach, the rules in a rule set are ordered in decreasing order of their priority, which can be defined in many ways {e.g., based on accuracy, coverage, total description length, or the order in which the rules are generated). An ordered rule set is also known as a decision list

Ex: Rules are ordered based on the way they are generated.

R1: (Give Birth = no)  $\wedge$  (Can Fly = yes)  $\rightarrow$  Birds

R2: (Give Birth = no)  $\wedge$  (Live in Water = yes)  $\rightarrow$  Fishes

R3: (Give Birth = yes)  $\wedge$  (Blood Type = warm)  $\rightarrow$  Mammals

R4: (Give Birth = no)  $\wedge$  (Can Fly = no)  $\rightarrow$  Reptiles

R5: (Live in Water = sometimes)  $\rightarrow$  Amphibians

| Name   | Blood Type | Give Birth | Can Fly | Live in Water | Class |
|--------|------------|------------|---------|---------------|-------|
| Turtle | Cold       | no         | No      | sometimes     | ?     |

### Unordered rules

- ☐ This approach allows a test record to trigger multiple classification rules and considers the consequent of each rule as a vote for a particular class.
- ☐ The votes are then tallied to determine the class label of the test record. The record is usually assigned to the class that receives the highest number of votes.
- ☐ In some cases, the vote may be weighted by the rule's accuracy.

### **Rule Ordering Scheme**

This ordering scheme ensures that every test record is classified by the "best" rule covering it and Individual rules are ranked based on their quality

Two ordering schemes:

- $\rightarrow$  Rule based ordering
- $\rightarrow$  Class based ordering

In rule based ordering, the rules are ordered based on their priority

In class based ordering, Rules that belong to the same class appear together

### Rule-based Ordering

(Refund=Yes) ==> No

(Refund=No, Marital Status={Single,Divorced},  
Taxable Income<80K) ==> No

(Refund=No, Marital Status={Single,Divorced},  
Taxable Income>80K) ==> Yes

(Refund=No, Marital Status={Married}) ==> No

### Class-based Ordering

(Refund=Yes) ==> No

(Refund=No, Marital Status={Single,Divorced},  
Taxable Income<80K) ==> No

(Refund=No, Marital Status={Married}) ==> No

(Refund=No, Marital Status={Single,Divorced},  
Taxable Income>80K) ==> Yes

### How to build a Rule Based Classifier?

There are two broad classes of methods for extracting classification rules:

- (1) **Direct method**, which extract classification rules directly from data
- (2) **Indirect method**, which extract classification rules from other classification models, such as decision trees and neural networks

#### Direct method (Sequential covering)

The sequential covering algorithm is often used to extract rules directly from data.

The algorithm extracts the rules one class at a time for data sets that contain more than two classes. For the vertebrate classification problem, the sequential covering algorithm may generate rules for classifying birds first, followed by rules for classifying mammals, amphibians, reptiles, and finally, fishes

#### Sequential Covering Algorithm steps:

1. Start from an empty rule
2. Grow a rule for a record
3. Remove training records covered by the rule
4. Repeat Step (2) and (3) until stopping criterion is met

---

### Algorithm 5.1 Sequential covering algorithm.

---

```

1: Let  $E$  be the training records and  $A$  be the set of attribute-value pairs,  $\{(A_j, v_j)\}$ .
2: Let  $Y_o$  be an ordered set of classes  $\{y_1, y_2, \dots, y_k\}$ .
3: Let  $R = \{ \}$  be the initial rule list.
4: for each class  $y \in Y_o - \{y_k\}$  do
5:   while stopping condition is not met do
6:      $r \leftarrow \text{Learn-One-Rule}(E, A, y)$ .
7:     Remove training records from  $E$  that are covered by  $r$ .
8:     Add  $r$  to the bottom of the rule list:  $R \rightarrow R \vee r$ .
9:   end while
10: end for
11: Insert the default rule,  $\{ \} \rightarrow y_k$ , to the bottom of the rule list  $R$ .
```

---

### Example of Sequential covering

Below figure demonstrates how the sequential covering algorithm works for a data set that contains a collection of positive and negative examples.

The rule R1, whose coverage is shown in Figure ii), is extracted first because it covers the largest fraction of positive examples.

All the training records covered by R1 are subsequently removed and the algorithm proceeds to look for the next best rule, which is R2.

### Rule growing

Two common strategies:

General-to-Specific

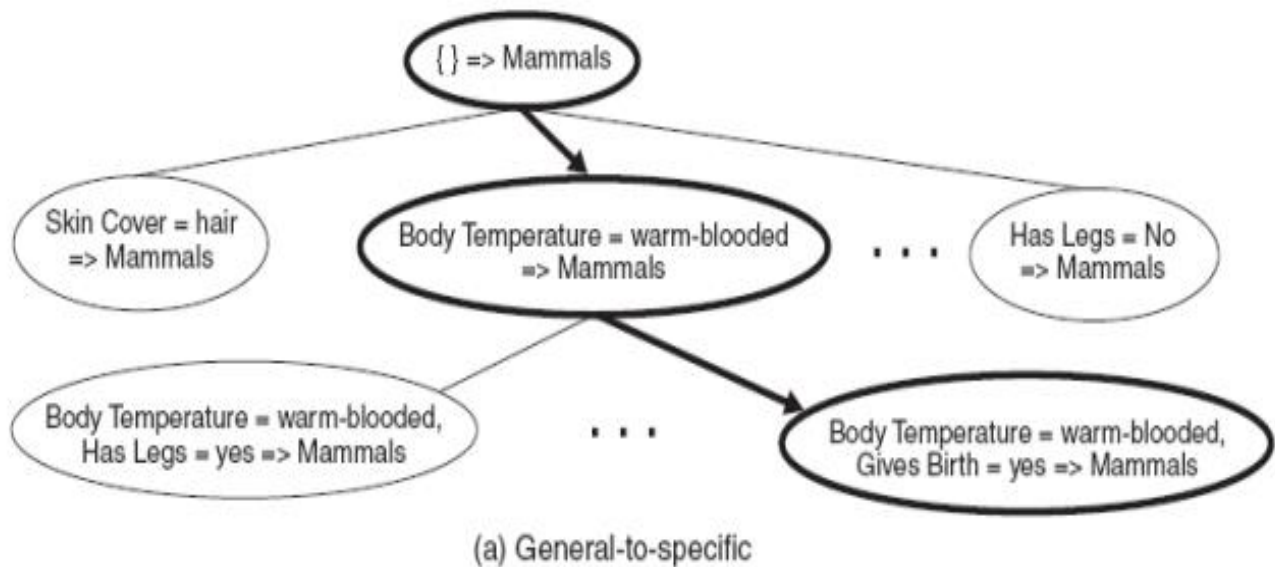
Specific-to-General

### General-to-Specific

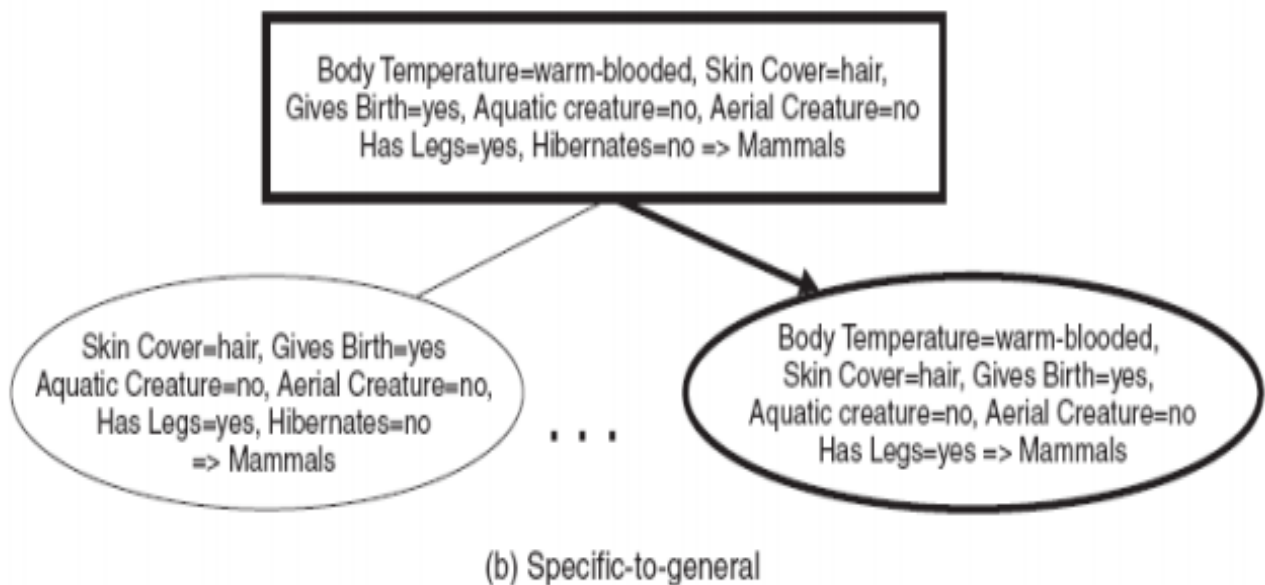
- An initial rule is created
  - $r: () \rightarrow y$
- Where left hand side is an empty set
- The rule has poor quality because it covers all the records in training set

→ New conjuncts are subsequently added to improve the rules quality

Example: Vertebrate classification problem



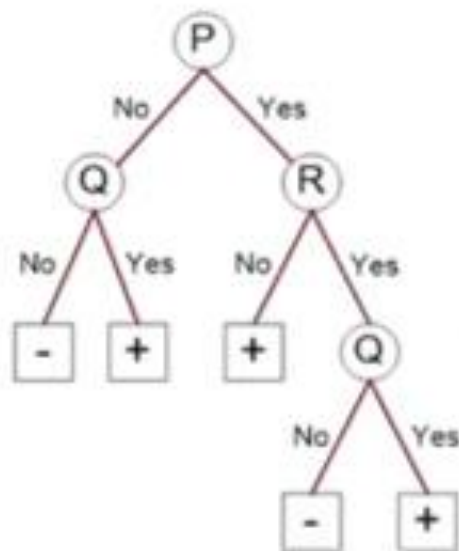
### Specific-to-General



### Indirect method

Generating a rule set from decision tree

Example:



### Rule Set

r1: (P=No,Q=No) ==> -  
 r2: (P=No,Q=Yes) ==> +  
 r3: (P=Yes,R=No) ==> +  
 r4: (P=Yes,R=Yes,Q=No) ==> -  
 r5: (P=Yes,R=Yes,Q=Yes) ==> +

### Advantages of rule based classifiers

As highly expressive as decision tree

Easy to interpret

Easy to generate

Can classify new instances rapidly

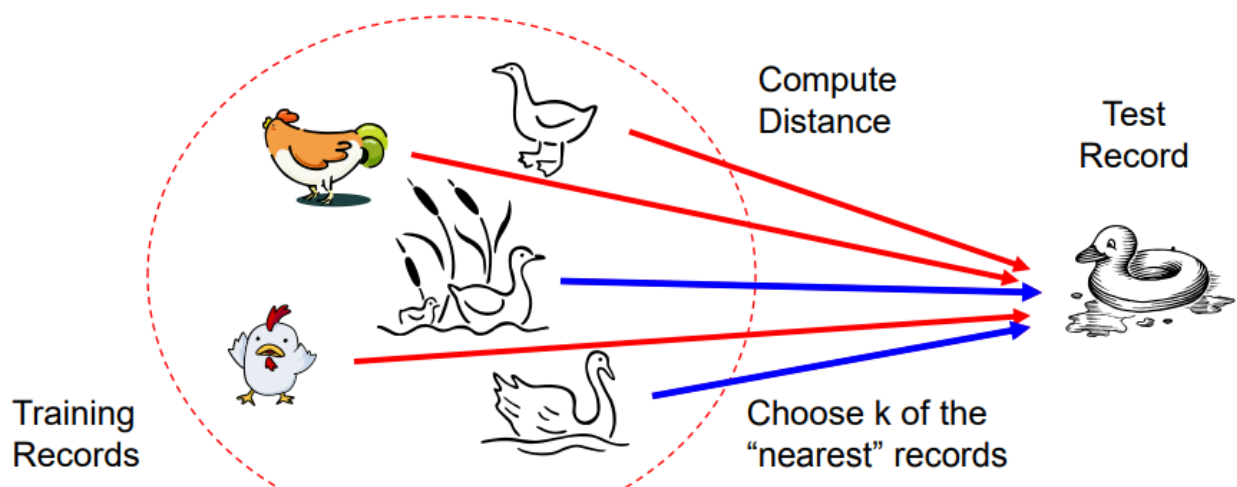
Performance comparable to decision tree

### 4.3.3 Nearest Neighbor classifier

Uses K- closest points (nearest neighbors) for performing classification

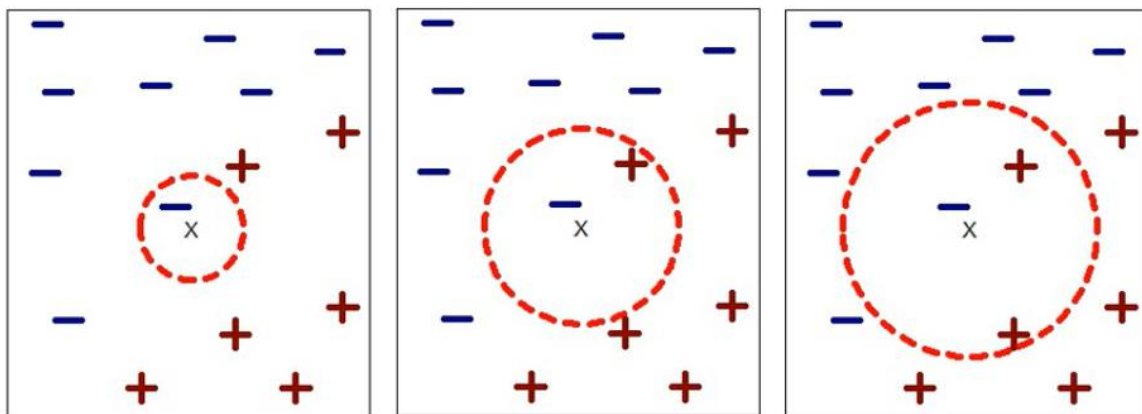
Basic Idea:

If it walks like a duck, quacks like a duck, then its probably a duck



## Requires three things

- The set of stored records
- Distance Metric to compute distance between records
- The value of  $k$ , the number of nearest neighbors to retrieve



(a) 1-nearest neighbor

(b) 2-nearest neighbor

(c) 3-nearest neighbor

**K-nearest neighbors of a record  $x$  are data points that have the  $k$  smallest distances to  $x$**

Above Figure illustrates the 1-, 2-, and 3-nearest neighbors of a data point located at the center of each circle. The data point is classified based on the class labels of its neighbors.

In the case where the neighbors have more than one label, the data point is assigned to the majority class of its nearest neighbors.

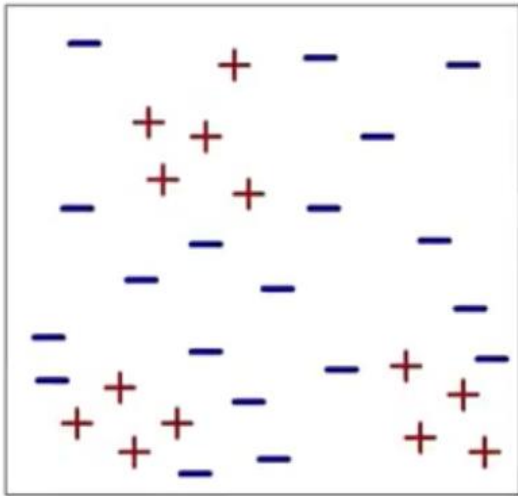
In Figure (a), the 1-nearest neighbor of the data point is a negative example. Therefore, the data point is assigned to the negative class.

If the number of nearest neighbors is three, as shown in Figure (c), then the neighborhood contains two positive examples and one negative example. Using the majority voting scheme, the data point is assigned to the positive class.

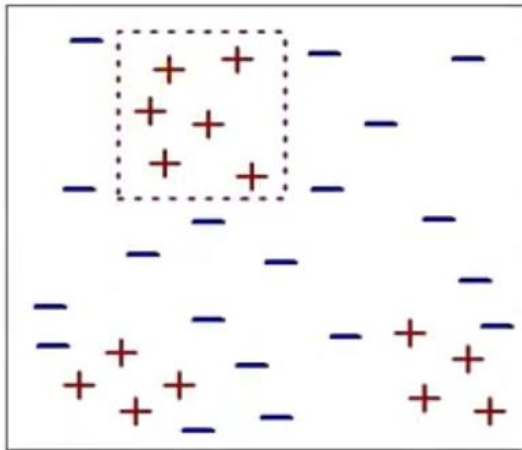
In the case where there is a tie between the classes (see Figure (b)), we may randomly choose one of them to classify the data point.

The preceding discussion underscores the importance of choosing the right value for  $k$ . If  $k$  is too small, then the nearest-neighbor classifier may be susceptible to over-fitting because of noise in the training data.

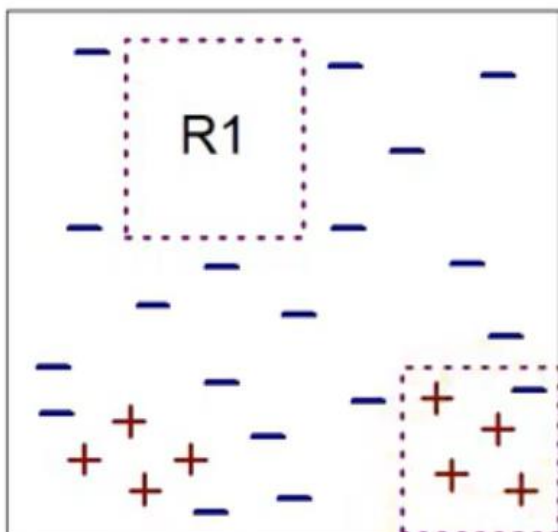
On the other hand, if  $k$  is too large, the nearest-neighbor classifier may misclassify the test instance because its list of nearest neighbors may include data points that are located far away from its neighborhood.



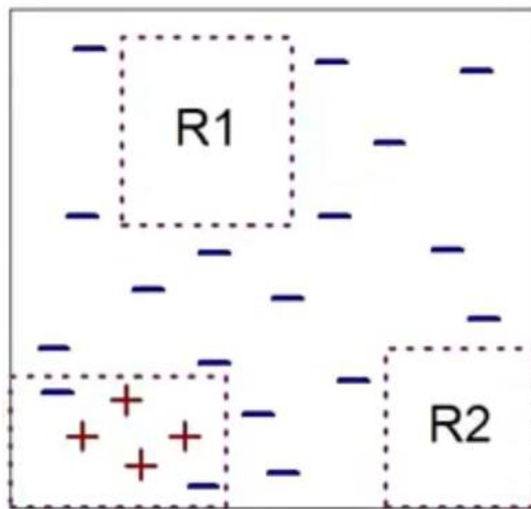
(i) Original Data



(ii) Step 1



(iii) Step 2



(iv) Step 3

## K-NN algorithm

---

**Algorithm 5.2** The  $k$ -nearest neighbor classification algorithm.

---

- 1: Let  $k$  be the number of nearest neighbors and  $D$  be the set of training examples.
  - 2: **for** each test example  $z = (\mathbf{x}', y')$  **do**
  - 3:   Compute  $d(\mathbf{x}', \mathbf{x})$ , the distance between  $z$  and every example,  $(\mathbf{x}, y) \in D$ .
  - 4:   Select  $D_z \subseteq D$ , the set of  $k$  closest training examples to  $z$ .
  - 5:    $y' = \operatorname{argmax}_v \sum_{(\mathbf{x}_i, y_i) \in D_z} I(v = y_i)$
  - 6: **end for**
- 

Once the nearest neighbor list is obtained, the test example is classified based on majority class of its nearest neighbour

- **Majority Voting:**

$$y' = \operatorname{argmax}_v \sum_{(\mathbf{x}_i, y_i) \in D_z} I(v = y_i)$$

where,  $v$  is a class label

$y_i$  is the class label for one of the nearest neighbors

$I$  is an indicator function that returns the value 1 if its argument is true and 0 otherwise.

## Example

| Name   | Acid Durability | Strength | Class |
|--------|-----------------|----------|-------|
| Type-1 | 7               | 7        | Bad   |
| Type-2 | 7               | 4        | Bad   |
| Type-3 | 3               | 4        | Good  |
| Type-4 | 1               | 4        | Good  |

Test-Data → acid durability=3, and strength=7, class=?

| Name   | Acid Durability | Strength | Class | Distance                             |
|--------|-----------------|----------|-------|--------------------------------------|
| Type-1 | 7               | 7        | Bad   | $\text{Sqrt}((7-3)^2 + (7-7)^2) = 4$ |
| Type-2 | 7               | 4        | Bad   | 5                                    |
| Type-3 | 3               | 4        | Good  | 3                                    |
| Type-4 | 1               | 4        | Good  | 3.6                                  |

Test-Data → acid durability=3, and strength=7, class=?

### Characteristics of Nearest-Neighbor Classifiers

Nearest-neighbor classification is part of a more general technique known as instance-based learning, which uses specific training instances to make predictions without having to maintain an abstraction (or model) derived from data.

Lazy learners such as nearest-neighbor classifiers do not require model building. However, classifying a test example can be quite expensive because we need to compute the proximity values individually between the test and training examples.

Nearest-neighbor classifiers make their predictions based on local information, whereas decision tree and rule-based classifiers attempt to find a global model that fits the entire input space.

Nearest-neighbor classifiers can produce arbitrarily shaped decision boundaries. Such boundaries provide a more flexible model representation compared to decision tree and rule - based classifiers

Nearest-neighbor classifiers can produce wrong predictions unless the appropriate proximity measure and data preprocessing steps are taken.

#### 4.3.4 Bayesian classifiers

**Bayes Theorem:** Bayes theorem is a statistical principle for combining prior knowledge of the classes with new evidence gathered from data.

Let X and Y be a pair of random variables

Their,

Joint probability,  $P(X=x, Y=y)$  refers to the probability that variable X will take on the value x and variable Y will take on the value y.

Conditional probability,  $P(Y=y | X=x)$  refers to the probability that the variable Y will take on the value y, given that the variable X is observed to have the value x

#### Formula of Bayes theorem

- The joint and conditional probabilities for X and Y are related in the following way:  

$$P(X, Y) = P(Y | X) \times P(X) = P(X | Y) \times P(Y)$$
- Rearranging the last two expressions leads to the following formula, known as the Bayes theorem:

$$P(Y | X) = \frac{P(X | Y)P(Y)}{P(X)}$$

#### Using Bayes theorem for classification

Consider the following data set:

| Tid | Home Owner | Marital Status | Annual Income | Defaulted Borrower |
|-----|------------|----------------|---------------|--------------------|
| 1   | Yes        | Single         | 125K          | No                 |
| 2   | No         | Married        | 100K          | No                 |
| 3   | No         | Single         | 70K           | No                 |
| 4   | Yes        | Married        | 120K          | No                 |
| 5   | No         | Divorced       | 95K           | Yes                |
| 6   | No         | Married        | 60K           | No                 |
| 7   | Yes        | Divorced       | 220K          | No                 |
| 8   | No         | Single         | 85K           | Yes                |
| 9   | No         | Married        | 75K           | No                 |
| 10  | No         | Single         | 90K           | Yes                |

**Figure 3.6.** Training set for predicting borrowers who will default on loan payments.

Suppose any test tuple comes with

marital status= married we need to classify them as defaulted borrower = yes or defaulted borrower= no

$P(\text{Defaulted borrower}=\text{No} | \text{marital status} = \text{married})$

$$\rightarrow \frac{P(\text{marital status} = \text{married} | \text{default borrower} = \text{No}) \times P(\text{default borrower} = \text{No})}{P(\text{marital status} = \text{married})}$$

$$\rightarrow \frac{4/7 \times 7/10}{4/10} = 1.00$$

$P(\text{Defaulted borrower}=\text{Yes} | \text{marital status} = \text{married})$

$$\rightarrow \frac{P(\text{marital status} = \text{married} | \text{default borrower} = \text{yes}) \times P(\text{default borrower} = \text{yes})}{P(\text{marital status} = \text{married})}$$

$$\rightarrow \frac{0/3 \times 3/10}{4/10} = 0$$

### Conclusion:

Since  $P(\text{Defaulted borrower}=\text{No} | \text{marital status} = \text{married}) > P(\text{Defaulted borrower}=\text{Yes} | \text{marital status} = \text{married})$

The test tuple with attribute marital status=married is classified as default borrower=No

### Naive Bayes classifier

A naive Bayes classifier estimates the class-conditional probability by assuming that the attributes are conditionally independent.

$$P(\mathbf{X} | Y = y) = \prod_{i=1}^d P(X_i | Y = y),$$

- Suppose we are given a test record with the following attribute set:  $X = (\text{Home Owner} = \text{No}, \text{Marital Status} = \text{Married}, \text{Annual Income} = \$120\text{K})$ .

$X = (\text{Home Owner} : \text{No}, \text{Marital Status} : \text{Married}, \text{Annual Income} : \$120\text{K})$

- $$P(X | \text{No}) = P(\text{Home Owner}=\text{No} | \text{No}) \\ \times P(\text{Status}=\text{Married} | \text{No}) \\ \times P(\text{Annual Income}=\text{120K} | \text{No}) \\ = 4/7 \times 4/7 \times 0.0072 = 0.0006$$
- $$P(X | \text{Yes}) = P(\text{Home Owner}=\text{No} | \text{Yes}) \\ \times P(\text{Status}=\text{Married} | \text{Yes}) \\ \times P(\text{Income}=\text{120K} | \text{Yes}) \\ = 1 \times 0 \times 1.2 \times 10^{-9} = 0$$

For the continuous variable

$$P(X_i = x_i | Y = y_j) = \frac{1}{\sqrt{2\pi}\sigma_{ij}} \exp^{-\frac{(x_i - \mu_{ij})^2}{2\sigma_{ij}^2}}.$$

$$\bar{x} = \frac{125 + 100 + 70 + \dots + 75}{7} = 110$$

$$s^2 = \frac{(125 - 110)^2 + (100 - 110)^2 + \dots + (75 - 110)^2}{7(6)} = 2975$$

$$s = \sqrt{2975} = 54.54.$$

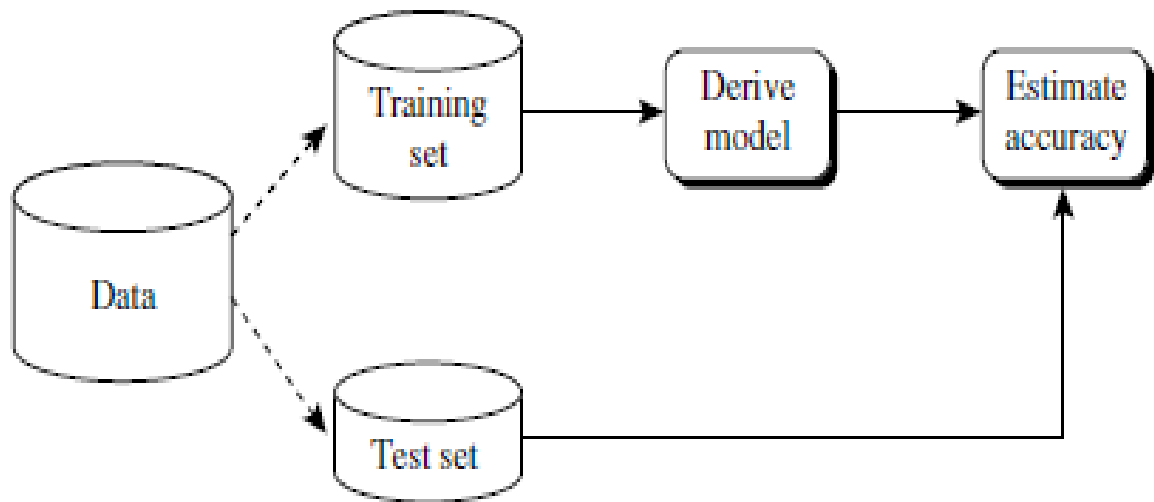
$$P(\text{Income}=120|\text{No}) = \frac{1}{\sqrt{2\pi}(54.54)} \exp^{-\frac{(120-110)^2}{2 \times 2975}} = 0.0072.$$

### Estimating predictive accuracy of classification method

The following are the different methods for estimating the accuracy of a method.

#### **Holdout method (Test sample method):**

- In this method, the given data are randomly partitioned into two independent sets, a *training set* and a *test set*. Typically, two-thirds of the data are allocated to the training set, and the remaining one-third is allocated to the test set. The training set is used to derive the model. The model's accuracy is then estimated with the test set



#### ***Random sub-sampling method:***

- **Random subsampling** is a variation of the holdout method in which the holdout method is repeated  $k$  times.
- The overall accuracy estimate is taken as the average of the accuracies obtained from each iteration.

#### ***K-fold cross validation method:***

- Training and testing is performed  $k$  times.
- In the first iteration, subsets  $D_2, \dots, D_k$  collectively serve as the training set to obtain a first model, which is tested on  $D_1$ ; the second iteration is trained on subsets  $D_1, D_3, \dots, D_k$  and tested on  $D_2$ ; and so on.
- The accuracy estimate is the overall number of correct classifications from the  $k$  iterations, divided by the total number of tuples in the initial data.

#### ***Boot strap method:***

- In this method, given a data set of size  $n$  a bootstrap sample is randomly selected uniformly with replacement (a sample may be selected more than once) and build a model.

#### **Other evaluation criteria for classification methods**

##### ***Speed:***

Speed involves not just the time or computation cost of constructing a model it also includes the time required to learn to use the model.

##### ***Robustness:***

Even if there are some data error, the classification method should be able to produce good results.

##### ***Scalability:***

As the data set increases the method should work efficiently for large data bases

##### ***Flexibility:***

Flexible enough for small modifications on the training data set

***Time complexity:***

Should not be exponential nor factorial. It should be in terms of log or linear or  $N^2$

**Interpretability**

It is desirable that the end-user be able to understand and gain insight from the results produced by the classification method